Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Explainable and LLM-Assisted CAN Bus Intrusion Detection: A Leakage-Audited Benchmark on the HCRL Car-Hacking Dataset

ROHIT R IYER[1] and SRIVIDYA P[2]

[1]Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, Karnataka 560059, India (e-mail: rohitriyer.ec23@rvce.edu.in)

[2]Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, Karnataka 560059, India (e-mail: srividyap@rvce.edu.in)

Corresponding author: Rohit R Iyer (e-mail: rohitriyer.ec23@rvce.edu.in).

This work was supported by Center of Excellence in e-mobility and researched on behalf of Department of Electronics and Communication Engineering, RV College of Engineering.

ABSTRACT The Controller Area Network (CAN) bus lacks authentication, encryption, and sender identification, which leaves in-vehicle networks exposed to message injection. Machine-learning intrusion detection systems report near-perfect accuracy on public benchmarks, yet two gaps persist: overlapping sliding-window datasets are rarely audited for train–test leakage, and detection outputs seldom carry evidence-grounded explanations that analysts can act upon. This paper presents a fully local, explainable intrusion detection pipeline evaluated on the complete HCRL Car-Hacking dataset of approximately 16.6 million messages. We quantify a leakage mode specific to overlapping windows: with 100 ms windows and a 50 ms stride, a row-wise split leaves a near-duplicate training window within 100 ms for 98.8% of test windows. A grouped split on non-overlapping 10 s time blocks removes all group overlap and reduces this contamination to 0.96%. Both splits yield the same held-out macro-F1 of 0.9990, so on this dataset the leakage did not inflate the headline metric; the grouped split establishes validity rather than changing the score. Twenty-five vehicle-agnostic behavioral features and a gradient-boosted tree classifier achieve an accuracy of 0.9992 and a per-class F1 above 0.998, including gear and RPM spoofing. Tree-based Shapley attributions feed a locally hosted language model that produces five-section incident reports, with a deterministic template fallback. A stream simulator and an eleven-panel dashboard demonstrate the pipeline without cloud services. We discuss limitations, notably single-vehicle data, and outline cross-dataset validation.

INDEX TERMS Automotive cybersecurity, controller area network, data leakage, explainable artificial intelligence, intrusion detection, large language models, SHAP, XGBoost.

I. INTRODUCTION

MODERN passenger vehicles integrate 70 to 100 or more electronic control units (ECUs) that manage powertrain, braking, steering, body, and infotainment functions, and most of them communicate over one or more Controller Area Network (CAN) buses [1]. CAN was standardized in the 1980s for deterministic, priority-arbitrated, real-time delivery and was never designed against an adversary. It is a broadcast, multi-master bus with no sender-identity field, no message authentication code, and no encryption. Any node with access to the bus, such as a compromised infotainment unit, a malicious telematics dongle, or a device attached to the OBD-II port, can inject arbitrary frames that every other ECU accepts as genuine. The remote compromise of a production Jeep Cherokee [2] and the security analysis of Koscher et al. [1] showed that CAN-level injection suffices for full vehicle takeover.

Because CAN cannot be authenticated without breaking compatibility with the installed ECU base, network-based intrusion detection has become the standard countermeasure: a passive monitor learns the statistical signature of legitimate traffic and flags deviations consistent with injection, flooding, or spoofing. A large body of work has evaluated machine-learning and deep-learning classifiers on the HCRL

Car-Hacking dataset [3], a capture from a Hyundai YF Sonata containing denial-of-service (DoS), fuzzing, gear-spoofing, and RPM-spoofing attacks, and reported accuracies approaching or exceeding 99.9% [4]–[6].

Results at this level warrant scrutiny on two grounds. First, most pipelines segment the message stream into fixed-length overlapping windows (typically 50–100 ms with a 50% stride), because per-message classification discards the timing and distributional context that separates attacks from normal traffic. Overlapping windows share about half of their messages with their temporal neighbours. A window-level train/test split, which is the default behaviour of common tooling such as `scikit-learn`'s `train_test_split`, therefore places near-duplicate windows on both sides of the split, a form of data leakage. Few of the CAN intrusion detection pipelines we reviewed document an audit of this effect or quantify its magnitude. Second, an alarm from a classifier is of limited operational value without an explanation of why the window was flagged, what attack mechanism it suggests, and what risk it poses, expressed in a form an analyst can act upon within seconds.

This paper addresses both gaps. Its contributions are as follows.

1) **A leakage-free evaluation methodology for overlapping-window CAN intrusion detection.** We characterize the leakage mode, introduce a grouped, non-overlapping time-block split, and verify it with two measured invariants: zero group overlap across training, validation, and test partitions, and a reduction of same-file sub-100 ms test contamination from 98.8% to 0.96%.
2) **A reproducible benchmark on the full HCRL Car-Hacking dataset.** Twenty-five generic behavioral features and an XGBoost classifier reach a held-out macro-F1 of 0.9990, with every class, including gear and RPM spoofing, above 0.998 F1. We measure the effect of the leakage fix directly (0.9990 under both the naive and the grouped split) rather than assuming it.
3) **A complete, local, explainable detection-to-response pipeline.** TreeSHAP [7] attributions computed from the deployed model are consumed by both a dashboard and a locally hosted language-model incident reporter that produces a five-section report, with a deterministic template fallback. A real-time replay simulator and an eleven-panel dashboard exercise the whole pipeline on laptop-class hardware without cloud dependencies.
4) **An explicit statement of scope.** We state what the results do not establish, namely cross-vehicle generalization, validation on physical hardware, and architectural novelty, so that the numbers can be interpreted and challenged correctly.

The remainder of the paper is organized as follows. Section II reviews related work. Section III states the threat model and describes the dataset. Section IV presents the methodology, including the leakage audit. Section V gives the experimental setup, and Section VI reports results. Section VII analyses the explanations, Section VIII discusses limitations, and Section IX concludes.

II. RELATED WORK

A. BENCHMARK DATASET AND DEEP-LEARNING DETECTORS

Seo et al. [3] released the HCRL Car-Hacking dataset and proposed GIDS, a generative adversarial network trained on normal traffic only, with CAN frames encoded as image-like inputs for a convolutional discriminator. The dataset has since become a de facto benchmark for in-vehicle intrusion detection. Song et al. [4] proposed a deep convolutional network inspired by Inception-ResNet and tuned for CAN traffic, and Hossain et al. [6] reported 99.995% detection accuracy with a long short-term memory (LSTM) model. Together these works set a near-ceiling performance bar for the dataset. Lokman et al. [8] review intrusion detection approaches for the automotive CAN bus more broadly.

B. CLASSICAL AND ENSEMBLE MACHINE LEARNING

Moulahi et al. [5] compared random forest, support vector machine, multilayer perceptron, and decision-tree classifiers on DoS, impersonation, and fuzzy attacks, and found tree ensembles highly effective at a fraction of the computational cost of deep models; this motivates the comparison of random forest, XGBoost, and LightGBM in this paper. Yagiz et al. [9] combined a variational-autoencoder knowledge-distillation pipeline with SHAP-based explanation on the same dataset and showed that gear- and RPM-spoofed messages account for only about 20–25% of an attack window's traffic and are therefore never the majority within the window. This observation informs the window-labeling rule used here (Section IV-B). Wang et al. [10] survey deep-learning-based in-vehicle detectors and note that inconsistent evaluation protocols limit comparability across studies.

C. GENERALIZATION BEYOND A SINGLE VEHICLE

HCRL Car-Hacking is a single-vehicle capture with a constrained attack set. The ROAD dataset and associated comparative studies [11], [12] evaluate methods, including LightGBM and random forest baselines, against more diverse and more carefully disguised attacks, and report a measurable performance gap relative to models trained on HCRL-style signatures. A recent survey [13] catalogues learning-based in-vehicle intrusion detection and observes that leakage-aware evaluation remains uncommon. Near-ceiling HCRL results, including those reported here, therefore should not be read as evidence of cross-vehicle or cross-dataset generalization.

D. EXPLAINABILITY AND LLM-ASSISTED SECURITY REPORTING

SHAP [7] unifies several feature-attribution methods under an additive, game-theoretic framework, and its TreeSHAP

variant computes exact attributions for tree ensembles efficiently. Habibzadeh et al. [14] reviewed 138 studies on large language models (LLMs) in security operations centre workflows, covering log analysis, alert triage, and incident reporting, and identified locally deployed, resource-constrained LLM reporting as an under-explored area. To the best of our knowledge, no prior work combines a leakage-audited CAN intrusion detection benchmark with SHAP-grounded, fully local LLM incident reporting in a single reproducible pipeline. Table 1 summarizes the positioning relative to closely related studies; the audit status listed for prior work reflects what each publication documents and not an independent re-audit.

III. THREAT MODEL AND DATASET

A. THREAT MODEL

We assume an attacker with electrical access to the CAN bus, for example through a compromised ECU, a malicious OBD-II dongle, or a telematics unit with remote code execution, who can inject arbitrary frames with arbitrary timing. This matches the model underlying the HCRL dataset and most of the literature reviewed above. Physical-layer attacks (bus jamming, voltage tampering) and attacks on cryptographic key material of higher-layer protocols such as AUTOSAR SecOC are out of scope. The work addresses detection of anomalous message-level behaviour resulting from unauthorized frame injection.

B. DATASET

We use the complete HCRL Car-Hacking dataset [3]: CAN logs recorded from a Hyundai YF Sonata through the OBD-II port, comprising approximately 16.6 million messages in five classes (Normal, DoS, Fuzzy, Gear, RPM). Every message in the four attack files carries a ground-truth flag read directly from the capture (`R` for regular, `T` for injected). Attacks occur as roughly 300 discrete bursts of 3–5 s per attack file, interleaved with genuine background traffic. The dataset is distributed in two raw formats, a keyword-tagged whitespace-delimited format for the normal file and a comma-delimited format for the attack files; the loader detects the format per line rather than per file so that mixed input cannot be silently misparsed. The dataset was chosen over OTIDS, the survival-analysis dataset, and ROAD [11] for comparability with prior work; evaluation on ROAD is left to future work.

Fig. 1 shows the class distribution of the windowed dataset used in the experiments (Section IV-A), totalling 190,422 windows.

IV. METHODOLOGY

Fig. 2 summarizes the pipeline, which consists of ten phases, each persisting its own artifacts (Parquet feature tables, the trained model, SHAP figures, metrics files) so that any stage can be re-run or audited independently: (1–2) ingestion and per-message labeling, (3) windowed feature extraction, (4) leakage-audited splitting and class balancing, (5) model training and selection, (6) held-out evaluation, (7) SHAP

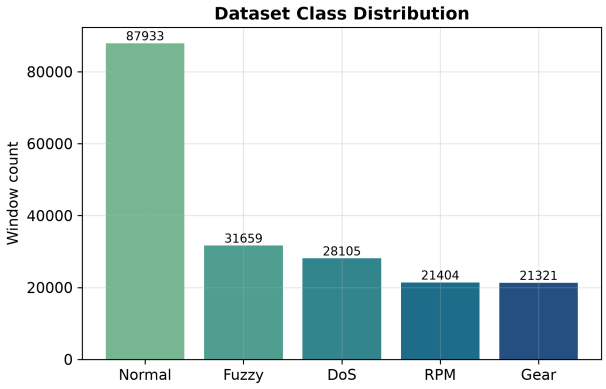


FIGURE 1. Class distribution of the windowed dataset used in the experiments (training-split Normal windows undersampled as described in Section IV-D).

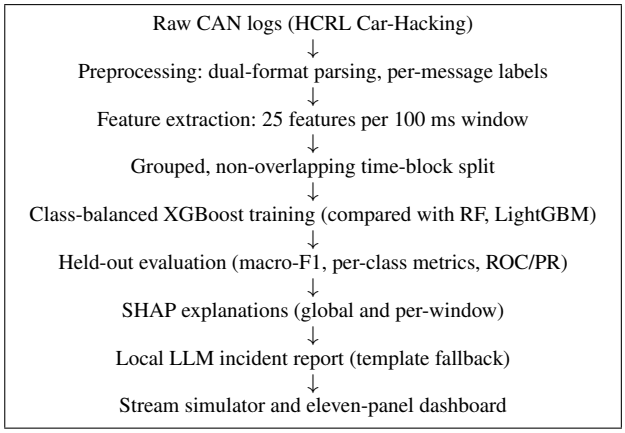


FIGURE 2. End-to-end pipeline. Each stage persists its artifacts, so the simulator and dashboard consume frozen, previously validated outputs.

explainability, (8) LLM-assisted incident reporting, (9) real-time stream simulation, and (10) an interactive dashboard. No stage refits on live data: the per-ID timing baseline and the trained classifier are frozen after training and reused unchanged by the simulator and dashboard.

A. WINDOWING AND FEATURE EXTRACTION

Messages are parsed, labeled from the ground-truth flag, and sorted by timestamp within each source file. Windows are never formed across concatenated files, because each file is an independent recording session. A sliding window of $W = 100$ ms with a stride of $S = 50$ ms (50% overlap) segments the stream; window k contains the messages $\{m_i : kS \leq t_i < kS + W\}$. Time-based windows keep features comparable between idle and busy bus states and allow the identical extraction code to be reused by the real-time simulator. Windows with fewer than three messages are discarded as statistically unreliable.

Twenty-five behavioral features are computed per window in six groups (Table 2). All are generic: an earlier design included two features derived from vehicle-specific target arbitration IDs, which were removed because CAN ID assign-

TABLE 1. Positioning Relative to Closely Related Work on the HCRL Car-Hacking Dataset Family

Work	Model	Reported metric	Leakage audit	Explainability	Incident reporting
Seo *et al.* [3] (2018)	GAN (GIDS)	≈98–100% accuracy	Not reported	None	None
Song *et al.* [4] (2020)	Deep CNN	Near-ceiling accuracy	Not reported	None	None
Hossain *et al.* [6] (2020)	LSTM	99.995% accuracy	Not reported	None	None
Moulahi *et al.* [5] (2021)	RF / SVM / MLP / DT	High accuracy (varies by attack)	Not reported	None	None
Yagiz *et al.* [9] (2024)	VAE with knowledge distillation	Competitive with ceiling	Not reported	SHAP (global)	None
This work	XGBoost (compared with RF, LightGBM)	**0.9990 macro-F1 (held out)**	**Audited and measured**	**SHAP (global and local)**	**Local LLM, five sections, template fallback**

TABLE 2. The 25 Generic Behavioral Features by Group

Group	Features
Timing (7)	mean, std, min, max inter-arrival time; message rate; burstiness; per-ID periodicity deviation from baseline
ID distribution (5)	unique-ID count; ID diversity ratio; dominant-ID ratio; Shannon ID entropy; unseen-ID ratio
DLC and entropy (5)	DLC mean, std, entropy; payload byte entropy; zero-payload ratio
Payload statistics (4)	payload mean and std; byte-0 mean; byte-7 mean
Hamming distance (3)	mean, max, std Hamming distance between consecutive same-ID payloads
Compound (1)	maximum per-ID message rate

ments are not standardized across vehicles and such features amount to a lookup table for one capture. The compound feature `max_id_msg_rate`, the message rate of the most frequent CAN ID in the window, replaces them without prior knowledge of any identifier. Shannon ID entropy is

$$H_{\mathrm{ID}} = -\sum_{j=1}^{n} p_j \log_2 p_j, \tag{1}$$

where p_j is the fraction of window messages carrying the j-th distinct ID.

A one-time calibration pass over the first 60 s of normal-only traffic builds a per-ID baseline (expected inter-arrival time and the set of known IDs). The baseline is persisted and never refit online, and it is used by the periodicity-deviation and unseen-ID features. The calibration slice is excluded from all windowing so that no training, validation, or test window overlaps the data that fit the baseline; otherwise the baseline would constitute a second leakage channel.

B. WINDOW LABELING

Labels follow an any-attack-present rule rather than majority vote. A window receives an attack label if any message of that class is present, and the most frequent attack class wins when a window straddles more than one:

$$y_k = \begin{cases} \arg\max_{c \in \mathcal{A}} n_{k,c}, & \text{if } \sum_{c \in \mathcal{A}} n_{k,c} > 0, \\ \text{Normal}, & \text{otherwise}, \end{cases} \tag{2}$$

where $\mathcal{A}$ is the set of attack classes and $n_{k,c}$ is the number of messages of class c in window k. The rule follows from the observation, consistent with [9], that gear and RPM spoofing inject at a normal-priority arbitration ID that shares bandwidth with background traffic and typically contributes only 20–25% of a window's messages even during a burst. DoS, by contrast, floods the highest-priority ID (`0x000`) and dominates its windows. Under majority-vote labeling, gear and RPM windows collapse toward the Normal class; the any-attack-present rule corrects this without altering any feature definition. Sensitivity to this labeling choice was not ablated (Section VIII).

C. LEAKAGE AUDIT AND GROUPED SPLIT

With $W = 100$ ms and $S = 50$ ms, each window shares about half of its raw messages with each immediate neighbour. A row-wise stratified split treats windows as independent rows and thereby scatters near-duplicates across partitions. We define the same-file sub-100 ms contamination of a test set $\mathcal{T}$ as

$$C = \frac{1}{|\mathcal{T}|} \sum_{w \in \mathcal{T}} c(w), \tag{3}$$

where $c(w) = 1$ if the training partition $\mathcal{D}_{\mathrm{train}}$ contains a window w' from the same source file, $f(w') = f(w)$, with $|t_w - t_{w'}| < 100$ ms, and $c(w) = 0$ otherwise; $f(\cdot)$ denotes the source file and t_w the window start time. Under the naive split, $C = 98.8\%$.

To remove this leakage structurally, windows are grouped into contiguous, non-overlapping 10 s time blocks within each source file, and each block is assigned atomically to one partition, stratified by the block's dominant window label. Because whole blocks move together, a window's temporal neighbours can fall in a different partition only at block boundaries. Two invariants verify the construction: (i) the groups of the training, validation, and test partitions are pairwise disjoint, confirmed on the full dataset, and (ii) the contamination C falls from 98.8% to 0.96%, the residual being attributable to block-boundary effects.

We then measured the effect of the leakage on the headline metric by training and evaluating both the naive and the grouped pipelines end to end, each on its own held-

out test partition. Both reach the same macro-F1 of 0.9990 (Section VI-A). Split strategy, random seed, and block size were not tuned in either direction.

D. CLASS BALANCING AND MODEL TRAINING

Grouping and splitting are performed on the full, unbalanced window table so that group atomicity is established before any resampling. The Normal class is then undersampled within the training partition only, down to $3.0\times$ the mean attack-class count; validation and test partitions are left untouched. The residual imbalance in the balanced training set (about 3.8:1 Normal to Gear) is handled with inverse-frequency sample weights,

$$w_c = \frac{N}{K\, n_c}, \tag{4}$$

where N is the number of samples, $K = 5$ the number of classes, and n_c the count of class c (the `balanced` scheme of `scikit-learn`). Weights are applied to both the training and the validation partitions, because early stopping on an unweighted validation loss would be dominated by the Normal class and could favour a model that underperforms on the minority attack classes.

Random forest, XGBoost [15], and LightGBM were trained under identical weighted conditions and compared on the validation partition. The deployed architecture was fixed in advance to XGBoost (`multi:softprob`, maximum depth 6, learning rate 0.1, up to 300 rounds, early stopping with patience 20 on multiclass log-loss), with the rule that a competitor must exceed XGBoost by more than 0.03 validation macro-F1 to override it; none did. Early stopping uses an explicit `EarlyStopping(..., save_best=True)` callback so that the persisted booster is pruned to the best validation iteration; the bare `early_stopping_rounds` argument halts training but retains post-optimal rounds.

E. SHAP EXPLANATIONS

A `shap.TreeExplainer` [7] wraps the deployed booster, so global and local explanations derive from the deployed model and not from a surrogate. For class c, the margin output decomposes additively as $f_c(\mathbf{x}) = \phi_{0,c} + \sum_j \phi_{j,c}(\mathbf{x})$. A single function, `explain_prediction`, returns the top-k signed attributions of a window together with the raw feature values; the dashboard and the LLM prompt builder both call it, so the two consumers cannot diverge.

F. LOCAL LLM INCIDENT REPORTING

The predicted class, its softmax confidence, and the top five signed SHAP features are passed to a locally hosted model, `qwen3:4b` served by Ollama, with no cloud dependency and no data leaving the machine, which reflects the data-sensitivity posture typical of automotive settings [14]. The prompt restricts the model to the supplied class, confidence, and features and forbids invented CAN IDs, byte values, or attacker identities. The report has five fixed sections (Incident Summary, Risk Level, Technical Analysis, Potential Impact, Recommended Mitigation) within a 200-word budget.

Two design decisions are safety-relevant. First, the risk level is computed from the softmax confidence p in code and overwrites the model's own wording:

$$\text{Risk}(p) = \begin{cases} \text{Critical}, & p \geq 0.90, \\ \text{High}, & 0.70 \leq p < 0.90, \\ \text{Medium}, & 0.50 \leq p < 0.70, \\ \text{Low}, & p < 0.50, \end{cases} \tag{5}$$

with windows predicted as Normal always labeled Informational. During development the free-form risk wording of `qwen3:4b` did not track the confidence reliably, and the override guarantees that the dashboard caption and the report body agree. Second, a required deterministic template fallback generates the same five-section structure, quoting the SHAP features verbatim, whenever Ollama is unreachable (not running, model not pulled, HTTP error, or timeout). The fallback is not a second LLM; the rest of the pipeline never depends on LLM availability. Both paths were exercised end to end on an NVIDIA RTX 4070 (8 GB VRAM), on which `qwen3:4b` loads fully on the GPU and produces a five-section report in low single-digit seconds.

G. STREAM SIMULATOR AND DASHBOARD

A stream simulator replays a recorded capture as a live message stream through the same extraction functions used in training (`_prepare_arrays` and `extract_window_features`), emitting and scoring each window as it arrives; this removes train/serve skew by construction. The persisted baseline is reused unchanged: refitting it on a live, possibly attacker-influenced stream would allow the model to gradually accept an ongoing attack as normal. In a representative DoS replay on the RTX 4070 machine, 61.7% of emitted windows were flagged as attack windows, consistent with the burst structure of the capture (Section III). An eleven-panel Streamlit dashboard (live feed, current prediction, attack status, confidence, attack statistics, confusion matrix, training metrics, loss curves, SHAP explanations, incident report, and attack timeline) contains no analysis logic of its own and only orchestrates the modules described above.

V. EXPERIMENTAL SETUP

All experiments use the grouped split of Section IV-C with 10 s blocks, a 70/15/15 train/validation/test ratio at the group level, and a fixed seed of 42 for preprocessing, splitting, balancing, and training. Table 3 lists the configuration and Table 4 the class composition of the partitions. The balanced training partition holds 123,928 windows; validation and test hold 33,247 windows each and are not modified by balancing. The best iteration was 114 of at most 300 rounds (575 trees across five classes). All test metrics are produced by a separate evaluation module, so the headline macro-F1 is a test-partition score and not the validation score used for model selection and early stopping. The pipeline

TABLE 3. Experimental Configuration

Parameter	Value
Window / stride	100 ms / 50 ms
Minimum messages per window	3
Calibration slice	first 60 s of normal traffic
Split unit / ratio	10 s blocks / 70:15:15
Normal undersampling (train only)	3.0× mean attack count
Sample weighting	balanced, train and validation
Objective	`multi:softprob`
Max depth / learning rate	6 / 0.1
Max rounds / patience	300 / 20 (log-loss)
Random seed	42

TABLE 4. Class Composition of the Balanced Training and the Held-Out Test Partitions (Windows)

Class	Train (balanced)	Test
DoS	19,560	4,249
Fuzzy	22,161	4,673
Gear	14,118	4,047
Normal	53,112	17,153
RPM	14,977	3,125
Total	**123,928**	**33,247**

runs on laptop-class hardware, with a single consumer GPU (NVIDIA RTX 4070, 8 GB) used only for the LLM stage and no cloud service at any stage.

VI. RESULTS

A. EFFECT OF THE LEAKAGE FIX

Table 5 summarizes the audit. The grouped split yields zero cross-partition group overlap and reduces same-file sub-100 ms test contamination by two orders of magnitude (98.8% to 0.96%), whereas the measured macro-F1 is identical under the two strategies (0.99904 in both cases, each on its own held-out test partition). On this benchmark the fix therefore delivers a defensible generalization estimate and not a different headline number. All remaining results use the grouped split exclusively.

B. MODEL COMPARISON

Table 6 compares the three model families on the validation partition. All fall within a 0.0002 macro-F1 band (0.99891–0.99911), which indicates that the 25-feature representation, and not the learner, sets the ceiling. Random forest leads XGBoost by 0.00006, far below the 0.03 margin required to override the pre-specified choice. XGBoost is retained for its shorter training time (3.0 s against 5.0 s for random forest and 4.9 s for LightGBM on the balanced training partition) and its support for best-iteration pruning.

C. HELD-OUT TEST PERFORMANCE

On the 33,247-window test partition the deployed model attains an accuracy of 0.9992, macro-precision 0.9992, macro-recall 0.9988, macro-F1 **0.9990**, and weighted-F1 0.9992.

TABLE 5. Leakage Audit: Measured Invariants Before and After the Grouped Split

Invariant	Naive split	Grouped split
Cross-partition group overlap	not controlled	**0**
Same-file <100 ms test contamination	98.8%	**0.96%**
Test macro-F1 (own held-out test)	0.9990	0.9990

TABLE 6. Validation-Split Comparison of Model Families (Balanced Sample Weights)

Model	Macro-F1	Weighted-F1	Train time (s)
Random forest	0.99911	0.99937	4.99
XGBoost (deployed)	**0.99905**	**0.99931**	**3.02**
LightGBM	0.99891	0.99922	4.88

Table 7 gives the per-class results. Every class exceeds 0.998 F1, including Gear (0.9983) and RPM (0.9990), which prior work identifies as the hardest because injected messages never dominate their windows [9]. The normalized confusion matrix (Fig. 3) has a diagonal proportion of 1.00 for every class at the plotted two-decimal resolution, and per-class F1 (Fig. 4) is near the ceiling for all five classes. One-vs-rest ROC and precision–recall curves (Figs. 5 and 6) give an AUC and average precision of 1.000 for every class at the reported precision.

D. TRAINING DYNAMICS

Fig. 7 shows the training and validation multiclass log-loss over boosting rounds. Validation loss reaches its minimum of 0.00451 at round 114; training stops after the 20-round patience window and the persisted model is pruned to the best iteration. Training and validation losses follow nearly identical trajectories. Fig. 8 shows gain-based importance from the final booster: the payload mean, byte-7 mean, `id_periodicity_dev`, and `unseen_id_ratio` account for most of the total gain, followed at a distance by `dominant_id_ratio` and `max_id_msg_rate`.

VII. EXPLAINABILITY ANALYSIS

A. GLOBAL ATTRIBUTIONS

Fig. 9 shows mean absolute SHAP values stacked by class. `unseen_id_ratio` has the largest total attribution, followed by `payload_byte7_mean`, `dominant_id_ratio`, and `id_periodicity_dev`. The class decomposition shows that these features serve different roles. The `unseen_id_ratio` contribution comes mainly from the Fuzzy class, which injects random identifiers that are largely absent from the normal-traffic baseline, with smaller contributions from Normal, Gear, and DoS. `payload_byte7_mean` acts almost entirely on the Gear and RPM classes, and `dominant_id_ratio` on Gear, RPM, and Normal. `id_periodicity_dev` is predominantly a Normal-class signal, consistent with legitimate periodic ECU traffic de-

TABLE 7. Per-Class Held-Out Test Metrics (33,247 Test Windows)

Class	Prec.	Rec.	F1	Support	ROC-AUC
DoS	0.9986	0.9998	0.9992	4,249	1.0000
Fuzzy	0.9998	0.9985	0.9991	4,673	1.0000
Gear	0.9995	0.9970	0.9983	4,047	1.0000
Normal	0.9992	0.9999	0.9996	17,153	1.0000
RPM	0.9990	0.9990	0.9990	3,125	1.0000
Macro avg.	**0.9992**	**0.9988**	**0.9990**	33,247	–

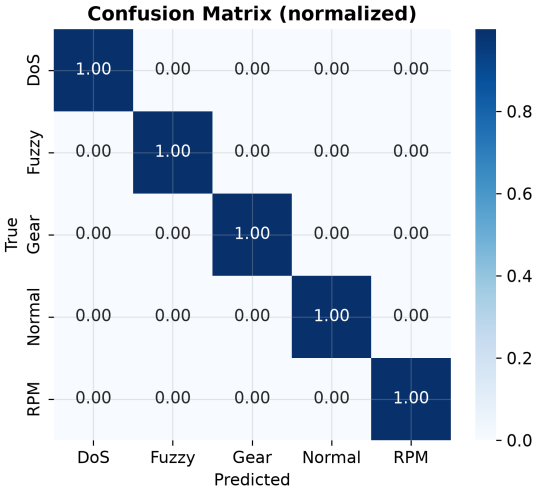


FIGURE 3. Normalized confusion matrix on the held-out test partition.

viating little from its calibrated inter-arrival times, whereas `payload_mean` and `zero_payload_ratio` act mainly on DoS.

B. PER-CLASS ATTRIBUTIONS

Fig. 10 gives the per-class SHAP summary plots, whose horizontal position shows the effect of a feature on the class margin and whose colour shows the feature value.

DoS. Low `payload_mean` and high `zero_payload_ratio` push strongly toward DoS, consistent with flooding by high-priority frames with all-zero payloads. Fuzzy. High `unseen_id_ratio` is the dominant positive driver, supported by high ID entropy and unique-ID count and by a low maximum per-ID rate, which together indicate many distinct, rarely repeated identifiers. Gear and RPM. For both spoofing classes, high `dominant_id_ratio` and `max_id_msg_rate` support the attack class, whereas a high unseen-ID ratio argues against it, because spoofed frames reuse legitimate identifiers. The byte-7 mean acts in opposite directions for the two classes (a high value pushes strongly away from Gear and toward RPM), so it serves as a discriminator between the two spoofing types. Normal. Low unseen-ID ratio, low dominant-ID ratio, and low periodicity deviation support Normal, while a high unseen-ID ratio is the strongest evidence against it.

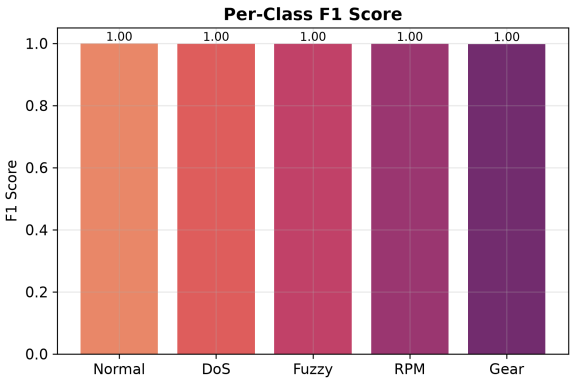


FIGURE 4. Per-class F1 on the held-out test partition. All classes exceed 0.998 (values are rounded to 1.00 in the bar labels).

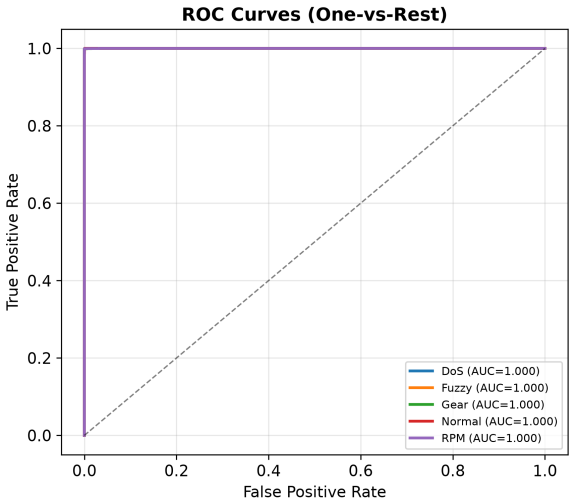


FIGURE 5. One-vs-rest ROC curves on the held-out test partition.

C. RELATION TO GAIN-BASED IMPORTANCE

The rankings of Figs. 8 and 9 differ: payload statistics lead in gain, whereas `unseen_id_ratio` leads in mean absolute SHAP value. The two measures answer different questions. Gain sums the loss reduction of the splits that use a feature, and features used for the early, coarse separations (such as the DoS payload signature) accumulate large gain. Mean absolute SHAP measures the average effect of a feature on the output margin, summed across classes. Features that rank highly under both measures (`unseen_id_ratio`, `id_periodicity_dev`, and the byte-7 mean) can be regarded as the most robust drivers of the model.

D. INCIDENT REPORTS

For a flagged window, the report generator receives the predicted class, the confidence, and the top five signed attributions from the same function used for Section IV-E. Both the LLM path and the template fallback were validated end to end (Section IV-F). The quality of the generated text was assessed qualitatively during development; a systematic evaluation of factual consistency with the supplied attributions is

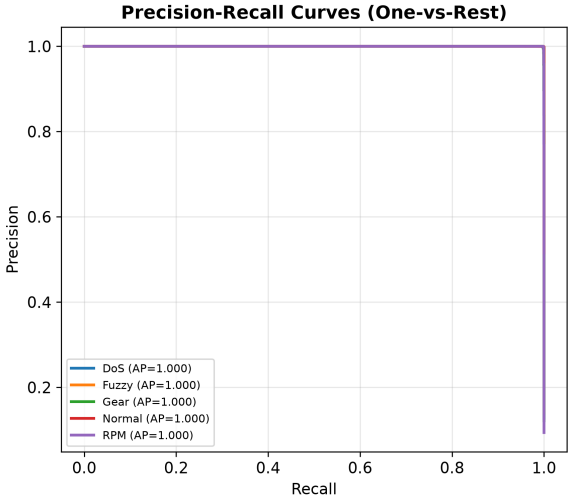
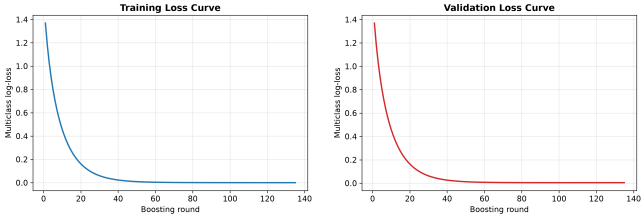


FIGURE 6. One-vs-rest precision–recall curves on the held-out test partition.



(a) (b)

FIGURE 7. Multiclass log-loss over boosting rounds: (a) training and (b) validation. The best validation iteration is 114; training halts 20 rounds later.

left to future work.

VIII. DISCUSSION

A. INTERPRETATION OF THE LEAKAGE FINDING

The audit is a methodological contribution and not an accuracy improvement. That the naive and the grouped split produce the same macro-F1 on this dataset shows that the 25 behavioral features separate the five classes almost perfectly even when no neighbouring window is shared across partitions. It does not show that overlapping-window leakage is harmless in general: whether leakage inflates a benchmark depends on the dataset, the features, and the model. A structurally enforced grouped split remains appropriate evaluation practice even when its effect on the score is negligible. The negative result is informative in itself, because it contradicts the implicit assumption in parts of the literature that such leakage would be material.

B. NEAR-CEILING SCORES

A macro-F1 of 0.9990 is consistent with the deep-learning results in Table 1, but it should not be over-interpreted. It reflects the fact that the four injected attack signatures in HCRL are almost linearly separable from normal traffic under the chosen features, not that in-vehicle intrusion detection is solved. More subtle or adaptive attacks, such as those in ROAD [11], [12], are known to be considerably harder for

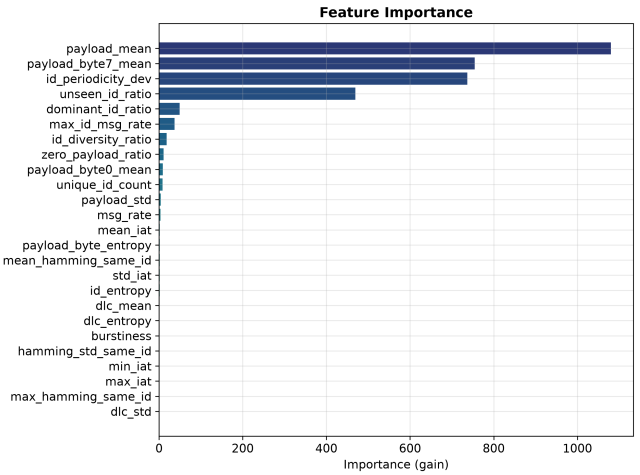


FIGURE 8. Gain-based feature importance of the final XGBoost booster.

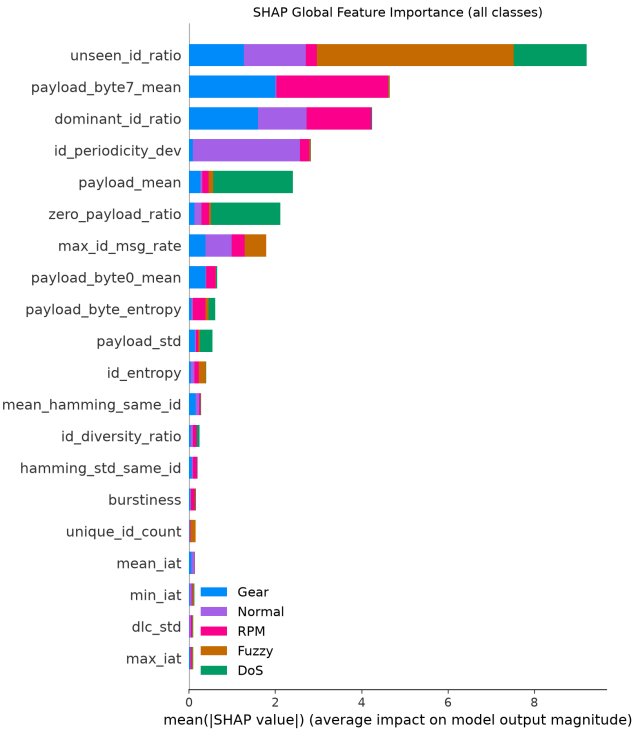


FIGURE 9. Global SHAP feature importance (mean absolute SHAP value) stacked by class for the deployed model.

models trained on HCRL-style signatures, and nothing here addresses that setting.

C. LIMITATIONS AND THREATS TO VALIDITY

- Single vehicle. All data come from one Hyundai YF Sonata. The features are vehicle-agnostic in their definition, but the SHAP analysis shows that several of the most discriminative features are payload statistics (payload mean and byte-7 mean), whose discriminative value may depend on the signal layout of this vehicle. Whether the representation transfers is untested.

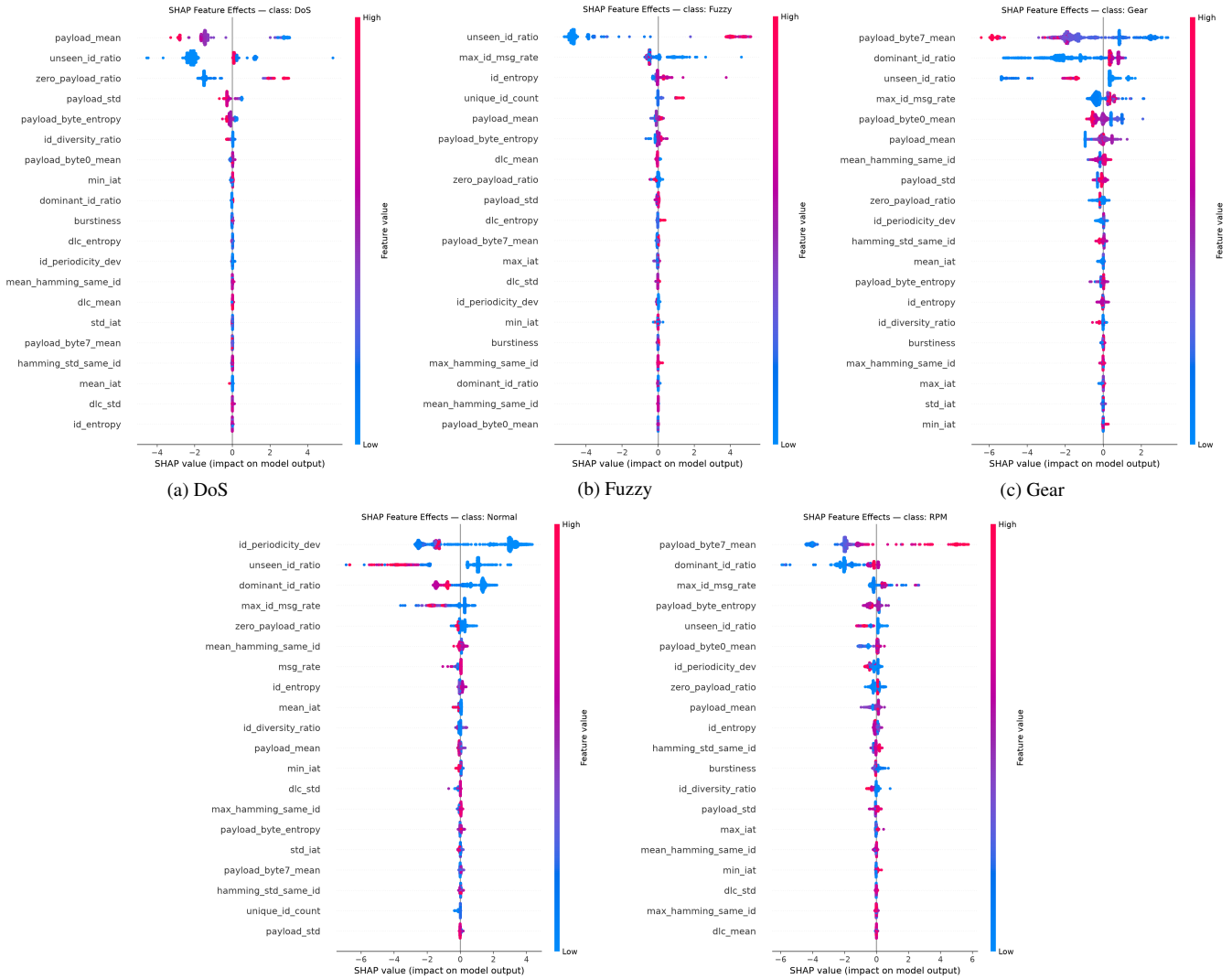


FIGURE 10. Per-class SHAP summary plots. Each point is one window; the horizontal axis is the SHAP value (impact on the class margin) and the colour encodes the feature value from low (blue) to high (red).

- Block-level, not session-level, split. The grouped split separates 10 s blocks within each recording; train and test blocks therefore come from the same capture sessions. Because the dataset provides one file per attack type, a file-level hold-out is not possible, which is a further reason for cross-dataset validation. The residual 0.96% contamination arises at block boundaries.
- Replayed captures. The simulator replays recorded data; no physical CAN bus or ECU was exercised, and real-time latency on in-vehicle hardware was not measured.
- Labeling scheme. The any-attack-present rule is one of several plausible choices (for example proportion-based soft labels), and its effect on the Gear and RPM results was not ablated.
- LLM risk wording. The free-form risk wording of `qwen3:4b` did not reliably match the confidence-calibrated level, which is why the programmatic override is used. Fine-tuning or few-shot calibration could close this gap.
- Scope of claims. The work does not claim architectural novelty; XGBoost on engineered features is a deliberate, well-understood choice, and the contribution lies in the evaluation methodology and the explainable, local detection-to-response pipeline.

IX. CONCLUSION

This paper presented a local, explainable CAN bus intrusion detection pipeline evaluated on the complete HCRL Car-Hacking dataset. A grouped time-block split removes overlapping-window leakage structurally and verifiably, reducing same-file sub-100 ms test contamination from 98.8% to 0.96% with zero group overlap, and a direct comparison shows that the leakage did not change the macro-F1 of 0.9990 on this dataset. A gradient-boosted classifier on 25 generic behavioral features achieves a per-class F1 above 0.998 for

all five classes, including gear and RPM spoofing. TreeSHAP attributions, computed once from the deployed model, ground both the dashboard and a locally hosted language-model incident reporter that falls back to a deterministic template, and a stream simulator with an eleven-panel dashboard demonstrates the full pipeline without cloud services.

Future work should first repeat the leakage-audited methodology on ROAD [11] and on captures from additional vehicles, to test whether the vehicle-agnostic feature design transfers and to permit file- or vehicle-level hold-out. Further directions include validation on a physical CAN testbed, native calibration of LLM risk wording through fine-tuning or few-shot prompting, a systematic evaluation of report faithfulness, and an ablation of the window-labeling rule against soft, proportion-based labels.

REFERENCES

[1] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in Proc. IEEE Symp. Security and Privacy, 2010, pp. 447–462.
[2] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," in Black Hat USA, 2015.
[3] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in Proc. 16th Annu. Conf. Privacy, Security and Trust (PST), 2018, pp. 1–6.
[4] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," Veh. Commun., vol. 21, p. 100198, 2020.
[5] T. Moulahi, S. Zidi, A. Alabdulatif, and M. Atiquzzaman, "Comparative performance evaluation of intrusion detection based on machine learning in in-vehicle controller area network bus," IEEE Access, vol. 9, pp. 99 595–99 605, 2021.
[6] M. D. Hossain, H. Inoue, H. Ochiai, D. Fall, and Y. Kadobayashi, "LSTM-based intrusion detection system for in-vehicle CAN bus communications," IEEE Access, vol. 8, pp. 185 489–185 502, 2020.
[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.
[8] S.-F. Lokman, A. T. Othman, and M. H. Abu-Bakar, "Intrusion detection system for automotive controller area network (CAN) bus system: A review," EURASIP J. Wireless Commun. Netw., vol. 2019, p. 184, 2019.
[9] M. A. Yagiz, P. MohajerAnsari, M. D. Pesé, and P. Goktas, "Transforming in-vehicle network intrusion detection: VAE-based knowledge distillation meets explainable AI," in Proc. 6th Workshop CPS & IoT Security and Privacy (CPSIoTSec), co-located with ACM CCS, 2024.
[10] Y. Wang et al., "Analysis of recent deep-learning-based intrusion detection methods for in-vehicle network," IEEE Trans. Intell. Transp. Syst., vol. 24, p. 1843, 2023.
[11] M. E. Verma, R. A. Bridges, M. D. Iannacone, S. C. Hollifield, P. Moriano, S. C. Hespeler, B. Kay, and F. L. Combs, "A comparative study of CAN-IDS using the ROAD dataset," Int. J. Inf. Secur., 2024.
[12] A. Alfardus and D. B. Rawat, "AI-driven intrusion detection systems (IDS) on the ROAD dataset: A comparative analysis for automotive controller area network (CAN)," arXiv preprint arXiv:2408.17235, 2024.
[13] V.-Q. Nguyen et al., "A survey of learning-based intrusion detection systems for in-vehicle network," arXiv preprint arXiv:2505.11551, 2025.
[14] A. Habibzadeh, F. Feyzi, and R. Ebrahimi Atani, "Large language models for security operations centers: A comprehensive survey," arXiv preprint arXiv:2509.10858, 2025.
[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794.